KROW

Documentación consolidada del proyecto

Estado funcional y técnico · Cumplimiento de actividades
Arquitectura · Configuración · Pruebas · Entrega

Corte documental	Versión de referencia	Alcance nativo
20 de octubre de 2026	Android 0.7.0-trial Código 12	Android iOS sin aceptación

Avance estimado: 83%
Las nueve actividades siguen parcialmente completadas. El desarrollo cuenta con evidencia automatizada; las pruebas físicas y el cierre de autenticación siguen pendientes.

Documento de consulta técnica y académica. Integra el estado más reciente de los repositorios móvil y backend/web, con las decisiones aprobadas y sus límites de verificación.

GUÍA / DOCUMENTACIÓN CONSOLIDADA

Índice y guía de lectura

Este PDF reúne la documentación vigente en una sola referencia. Empieza por el resumen y las nueve actividades para conocer el avance; usa los capítulos técnicos para revisar implementación, configuración, evidencia y aceptación.

Resumen y alcance del corte . . . 3

Avance de los nueve compromisos . . . 4

Qué se cumple en cada actividad . . . 5

Arquitectura, navegación y datos . . . 14

Solicitudes, confirmaciones y chat . . . 16

Programación, recurrencia e invitaciones . . . 18

Lectura sin conexión y recuperación . . . 20

Notificaciones, Firebase y recordatorios . . . 22

Enlaces y recuperación de cuenta . . . 24

Instalación, ejecución y configuración . . . 25

Pruebas, comprobaciones y límites . . . 27

Entrega publicada y paquete Android . . . 29

Matriz de aceptación pendiente . . . 30

Orden recomendado y cierre académico . . . 31

Fuentes, archivos y glosario . . . 32

Cómo interpretar el documento
“Implementado” describe código integrado. “Verificado” precisa el entorno de prueba. “Pendiente” identifica una validación o configuración sin evidencia suficiente. Una pantalla o un build por sí solos no acreditan funcionamiento completo.

01 / DOCUMENTACIÓN CONSOLIDADA

Resumen y alcance del corte

Fuentes consolidadas: [F01], [F02], [F03], [F04], [F09]

KROW cuenta con una base funcional para compartir viajes entre conductores y pasajeros. Las últimas etapas integraron bandeja de solicitudes, confirmación al pasajero, corrección del viaje activo vacío y del chat, permiso de notificaciones al abrir, publicación de varias fechas, invitaciones compartidas y recordatorios de salida.

> **Estado general**
> Avance aproximado ponderado: 83,3%, comunicado como 83%. Ninguna de las nueve actividades se declara cerrada al 100%: aún falta aceptación física, configuración/recorrido de Auth y cierre de evidencias.

Lo que ya está disponible

- API NestJS desplegada en Railway y páginas de respaldo de enlaces en Vercel.
- APK privado 0.7.0-trial generado, con firma y contenido comprobados.
- Flujos y datos conectados: publicación, búsqueda, reservas, solicitudes/respuestas, perfiles, chat e historial.
- Lectura parcial sin conexión y recuperación segura de publicaciones interrumpidas.
- Pruebas de reglas, HTTP, componentes, navegación y permisos simulados; comprobaciones contra servicios publicados.

Decisiones confirmadas

Tema	Alcance elegido
Plataforma	Android primero. iOS no está construido ni certificado.
Recurrencia	Publicar varias fechas con vista previa y confirmación. Hasta cuatro semanas, cada salida administrada por separado.
Invitaciones	Enlace compartido. El pasajero solicita y el conductor aprueba.
Offline	Lectura parcial de datos propios y recuperación explícita; operaciones de negocio requieren conexión.
Arquitectura	Conservar React Native, React Query, NestJS, Supabase, FCM y diseño vigente.

Regla de consolidación

Se da prioridad a la etapa 0.7.0 y su evidencia conjunta. Los informes 0.5.x y 0.6.0 conservan contexto histórico; sus flags desactivados, incidencias de despliegue y tareas posteriores ya implementadas no se presentan como estado actual. Las guías se consolidan sin reproducir claves privadas, tokens, contraseñas ni valores secretos.

Este PDF compila evidencia existente; su generación no ejecuta pruebas nuevas de la app ni convierte en verificadas las comprobaciones físicas pendientes.

02 / DOCUMENTACIÓN CONSOLIDADA

Avance de los nueve compromisos

Fuentes consolidadas: [F01], [F03], [F09] y estimación comunicada en esta conversación

Los porcentajes son una estimación de cumplimiento por requisitos y evidencia. No se calcularon contando archivos ni a partir de horas consumidas. Las semanas y horas pertenecen al plan académico original.

Actividad	Semanas	Horas	Avance	Estado
1. Requerimientos UX, flujos y arquitectura de navegación	1-3	60	85%	Parcial
2. Estado global y estructura de la aplicación móvil	2-4	50	90%	Parcial
3. Navegación móvil y mecanismos de Deep Linking	3-6	80	85%	Parcial
4. Interfaces, componentes reutilizables y experiencia visual	4-8	120	90%	Parcial
5. Mapas interactivos, calendarios y programación de rutas	5-9	60	80%	Parcial
6. Funcionamiento offline parcial, persistencia y sincronización	6-10	55	75%	Parcial
7. Notificaciones push y eventos de usuario	8-11	35	70%	Parcial
8. Rendimiento, accesibilidad y pruebas de interfaz	10-14	25	75%	Parcial
9. Validaciones finales, documentación y entrega del frontend	12-16	15	80%	Parcial

Cálculo del indicador global

El plan original suma 500 horas. Para mantener la importancia relativa de sus compromisos, el indicador comunicado pondera el porcentaje de cada actividad por las horas originales: suma(horas × porcentaje) / 500 = 83,3%. El promedio simple de las nueve actividades sería 81,1%. Ambos son aproximaciones; no son una medición de horas restantes.

Criterios de cierre

Clasificación	Significado
Completada	Todos los requisitos importantes implementados y evidencia suficiente de funcionamiento.
Parcialmente completada	Hay avances integrados, pero faltan configuraciones, validaciones o entregables.
Pendiente	El requisito o entregable todavía no está implementado.
No verificable	Hay código o preparación relacionada, sin evidencia suficiente para confirmar funcionamiento.

Todas las actividades se mantienen en “Parcialmente completada”. Dentro de ellas, la recepción push, App Links, GPS y almacenamiento físico se consideran pendientes de verificación, aunque tengan implementación y pruebas simuladas.

03 / DOCUMENTACIÓN CONSOLIDADA

Qué se cumple en cada actividad

Fuentes consolidadas: [F01], [F02], [F03], [F04], [F06], [F09]

Cada ficha separa lo que ya se desarrolló de la aceptación que falta. Las rutas son referencias a los repositorios locales; sus raíces completas aparecen al final.

Actividad 1 · Requerimientos UX, flujos y arquitectura de navegación

Avance estimado	Estado	Plan original
85%	Parcialmente completada	Semanas 1-3 · 60 horas

KROW ya tiene organizados los pasos que siguen conductores y pasajeros, desde entrar a la aplicación hasta consultar y completar sus viajes.

Ya implementado

- Recorridos de autenticación, publicación, búsqueda, solicitud, respuesta, viaje programado, seguimiento, historial, perfil y chat.
- Pantallas organizadas mediante stacks y pestañas, con separación de acceso por sesión y comprobación de conductor.
- Estrategia de enlaces para viajes, solicitudes e invitaciones compartidas; el destino se conserva durante el login.
- Arquitectura, decisiones de alcance y trazabilidad funcional documentadas.

Qué falta para cerrarla

Ejecutar los recorridos completos de aceptación y contrastar los requisitos no funcionales con el comportamiento real: claridad, retorno, permisos, tamaños de pantalla y recuperación de errores.

Por qué no se considera completada

Los recorridos principales y su organización están implementados; falta la aceptación integral y evidencia de uso nativo para considerar cerrado el compromiso.

Referencias principales de código y documentación

src/app/navigation/RootNavigator.tsx

src/app/navigation/MainNavigator.tsx

src/app/navigation/MainTabs.tsx

docs/architecture.md

Actividad 2 · Estado global y estructura de la aplicación móvil

Avance estimado	Estado	Plan original
90%	Parcialmente completada	Semanas 2-4 · 50 horas

La aplicación organiza los datos de usuarios, viajes y reservas, y actualiza las pantallas cuando cambia esa información.

Ya implementado

- React Query para consultas, caché, invalidación y refrescos; hooks locales para formularios e interacción.
- Separación entre pantallas, componentes visuales, servicios HTTP y reglas de negocio; reutilización de componentes y temas.
- Consultas vinculadas al actor que las inició y limpieza al salir o cambiar de cuenta.
- Persistencia protegida de avisos vistos, consulta offline y lotes de publicaciones interrumpidas.
- Pruebas de cambio de cuenta, respuestas tardías, concurrencia y recuperación. No fue necesaria una migración a Redux o Zustand.

Qué falta para cerrarla

Confirmar aislamiento, restauración y consistencia usando Keystore y ciclo de vida reales, incluidos cambios de cuenta durante peticiones y almacenamiento fallido.

Por qué no se considera completada

La base interna y sus protecciones están implementadas y probadas; el cierre depende principalmente de su aceptación con almacenamiento nativo y sesión real.

Referencias principales de código y documentación

src/core/api/apiClient.ts

src/features/home/hooks/useCurrentUserRole.ts

src/features/ride/hooks/invalidateRideLists.ts

src/features/pilot/bookingEvents.ts

src/features/ride/api/publicationBatch.ts

Actividad 3 · Navegación móvil y mecanismos de Deep Linking

Avance estimado	Estado	Plan original
85%	Parcialmente completada	Semanas 3-6 · 80 horas

Puedes moverte por las funciones principales y está preparado abrir directamente un viaje, una solicitud o una invitación desde un enlace.

Ya implementado

- Separación entre autenticación y navegación principal, con control de acceso de conductor.
- Acceso a publicación, búsqueda, viajes, reservas, perfil, historial y chat; protección al abandonar una publicación sin guardar.
- Enlaces HTTPS al dominio aprobado con UUID válido; una reserva concreta usa bookingId y una invitación usa invite=1.
- El destino se retiene hasta que termine el login; el backend vuelve a autorizar la consulta.
- Asociación Android y páginas web de respaldo publicadas para viajes y callback de acceso.

Qué falta para cerrarla

Verificar App Links con el sistema operativo, retroceso y apertura en frío/caliente. Guardar el callback de Supabase y probar recuperación/confirmación de cuenta mediante correo real.

Por qué no se considera completada

La lógica y el respaldo publicados tienen pruebas; la asociación y apertura reales del SO, junto con el recorrido completo de Auth, aún no están acreditados.

Referencias principales de código y documentación

src/app/navigation/RootNavigator.tsx

src/features/ride-runtime/deepLinks.ts

android/app/src/main/AndroidManifest.xml

apps/web/src/lib/mobile-links.ts

apps/web/public/.well-known/assetlinks.json

Actividad 4 · Interfaces, componentes reutilizables y experiencia visual

Avance estimado	Estado	Plan original
90%	Parcialmente completada	Semanas 4-8 · 120 horas

Ya existen las pantallas, formularios y controles para usar KROW, con avisos claros de solicitudes, confirmaciones, cargas y errores.

Ya implementado

- Pantallas principales conectadas a la API, formularios y componentes de botones, listas, tarjetas, perfiles y estados.
- Sistema de temas claro/oscuro y componentes compartidos; respeto de movimiento reducido en las animaciones existentes.
- Solicitudes prioritarias en Inicio, hoja animada y pantalla dedicada para revisar, aceptar o rechazar.
- Confirmación destacada al pasajero con viaje, conductor, vehículo, parada y horario según datos disponibles.
- Chat cronológico sin inversión, paginación y comportamiento de scroll ajustado; respuestas visuales a las acciones.

Qué falta para cerrarla

Validar visualmente tamaños de pantalla y fuente, temas, teclado, lectores de pantalla, áreas táctiles y animaciones en Android; corregir lo que impida operar los flujos.

Por qué no se considera completada

La interfaz funcional y los patrones visuales están desarrollados, pero una prueba de renderizado simulado no certifica apariencia ni usabilidad físicas.

Referencias principales de código y documentación

src/shared/components/ui-v2/

src/shared/theme/ThemeProvider.tsx

src/features/pilot/BookingNoticeCard.tsx

src/features/pilot/BookingEventsHost.tsx

src/features/pilot/ChatScreen.tsx

Actividad 5 · Mapas interactivos, calendarios y programación de rutas

Avance estimado	Estado	Plan original
80%	Parcialmente completada	Semanas 5-9 · 60 horas

Los conductores pueden publicar fechas y paradas; los pasajeros pueden encontrar viajes y solicitar su lugar. También se pueden preparar varias salidas de una vez.

Ya implementado

- Mapas integrados y cálculo de rutas mediante el backend; catálogo de avenidas y paradas publicadas por el conductor.
- Publicación futura con fecha, horario, vehículo, cupo y aportación; búsqueda por destino y filtros de fechas.
- Selección de subida/bajada, solicitud y aprobación; validaciones de vigencia y capacidad en servidor.
- Recurrencia finita de una a cuatro semanas, días seleccionados, vista previa y confirmación de hasta 28 salidas.
- Recuperación del lote interrumpido con identificadores por salida; permisos y código nativo de seguimiento GPS existentes.

Qué falta para cerrarla

Ejecutar recorrido GPS real con dos participantes y verificar mapas, paradas y estados. Probar selectores nativos, lote recurrente y cambios de ruta con conectividad real; el runtime avanzado permanece desactivado.

Por qué no se considera completada

La lógica de viajes y programación está desarrollada; ubicación, mapas y comportamiento en recorrido real siguen siendo una parte importante de la aceptación.

Referencias principales de código y documentación

src/features/ride/screens/driver/PublishRideScreen.tsx

src/features/ride/screens/passenger/RequestRideScreen.tsx

src/features/ride/domain/recurrence.ts

src/features/pilot/nativeTracking.ts

apps/api/src/modules/pilot/

Actividad 6 · Funcionamiento offline parcial, persistencia y sincronización

Avance estimado	Estado	Plan original
75%	Parcialmente completada	Semanas 6-10 · 55 horas

Cuando falla internet, KROW puede mostrar parte de la información que ya habías consultado y actualizarla al volver la conexión.

Ya implementado

- Perfil y primera página de listas propias disponibles desde copias protegidas por cuenta y API.
- Vigencia de lectura de 24 horas, hasta 30 resúmenes por grupo y límite de tamaño por registro.
- Recuperación tras errores de red o timeout, indicador de copia y revalidación en primer plano/al reanudar.
- Bloqueo de acciones operativas basadas en datos offline desactualizados; eliminación de copias ante autorización denegada.
- Idempotencia de publicación y persistencia del lote para retomarlo explícitamente sin repetir salidas confirmadas.

Qué falta para cerrarla

Validar modo avión, caducidad, restablecimiento de red, Keystore y aislamiento real entre cuentas. Documentar los resultados observados y corregir fallos, conservando el alcance parcial acordado.

Por qué no se considera completada

Se cumple una estrategia parcial y delimitada, no una app entera disponible offline. Faltan pruebas nativas de almacenamiento y reconexión; las escrituras siguen requiriendo servidor.

Referencias principales de código y documentación

src/core/offline/readOnlyCache.ts

src/features/pilot/readOnlyQueries.ts

src/core/api/apiClient.ts

src/features/ride/api/publicationBatch.ts

docs/offline-readonly.md

Actividad 7 · Notificaciones push y eventos de usuario

Avance estimado	Estado	Plan original
70%	Parcialmente completada	Semanas 8-11 · 35 horas

KROW está preparado para pedir permiso y avisar sobre solicitudes, respuestas y recordatorios; todavía falta comprobar la llegada real en teléfonos.

Ya implementado

- Proyecto Firebase Android real configurado y credenciales de servidor alojadas en la API; flags habilitados para pruebas privadas.
- Solicitud de permiso al abrir en Android 13+ y registro automático tras login si está concedido; Perfil conserva activación y ajustes.
- Tokens cifrados, cuenta destinataria, caducidad, outbox con reintentos y deduplicación.
- Eventos de reservas, respuestas, cancelaciones y chat, más recordatorios dentro de los 30 minutos previos a partir.
- Payload con reserva concreta y navegación preparada; FCM HTTP v1 validate_only respondió 200 sin entregar mensajes.

Qué falta para cerrarla

Registrar un Android real y validar recepción/presentación/apertura en primer plano, segundo plano y proceso cerrado, incluida negativa, revocación, cambio de cuenta y recordatorios obsoletos.

Por qué no se considera completada

La infraestructura y la lógica están preparadas, pero la entrega remota real es central para esta actividad y aún no tiene evidencia física suficiente.

Referencias principales de código y documentación

src/features/pilot/notificationPermission.ts

src/features/pilot/pilotPush.ts

android/app/src/main/java/com/krownmobileapp/runtime/KrowMessagingService.kt

apps/api/src/modules/pilot/pilot.notifications.ts

apps/api/src/modules/pilot/pilot.reminders.ts

Actividad 8 · Rendimiento, accesibilidad y pruebas de interfaz

Avance estimado	Estado	Plan original
75%	Parcialmente completada	Semanas 10-14 · 25 horas

Ya se corrigieron errores importantes y se hicieron pruebas automáticas para comprobar las reglas y los flujos; falta observar cómo se comporta en celulares reales.

Ya implementado

- 269 pruebas móviles, 197 de API, 9 E2E y 57 web aprobadas, con alcance y dobles de prueba documentados.
- Tipos, compilación API y Android y lint sin errores; se mantienen advertencias de lint y avisos de renderer en suites existentes.
- Regresiones de doble pulsación, concurrencia, cancelación, pérdida de red, reintentos y cambio de cuenta.
- Corrección de respuesta vacía cuando no hay viaje activo y del chat invertido; orden/paginación ajustados.
- Etiquetas accesibles en controles seleccionados, temas y movimiento reducido; compilación Hermes y APK no depurable verificados.

Qué falta para cerrarla

Medir fluidez, memoria, listas y mapas en Android; validar teclado/scroll, texto ampliado, lector de pantalla, áreas táctiles y estabilidad del recorrido con dos participantes.

Por qué no se considera completada

La batería automatizada reduce riesgos lógicos, pero no equivale a un perfilado de rendimiento ni a aceptación de accesibilidad o estabilidad en hardware real.

Referencias principales de código y documentación

__tests__/

apps/api/src/modules/pilot/pilot.integration.spec.ts

apps/api/test/pilot.e2e-spec.ts

src/features/pilot/ChatScreen.tsx

src/shared/theme/ThemeProvider.tsx

Actividad 9 · Validaciones finales, documentación y entrega del frontend

Avance estimado	Estado	Plan original
80%	Parcialmente completada	Semanas 12-16 · 15 horas

Ya hay instrucciones, documentos del funcionamiento, resultados de pruebas y un APK para instalar; falta registrar la aceptación final.

Ya implementado

- Arquitectura, navegación, configuración, Firebase, enlaces, consulta offline y decisiones funcionales documentadas.
- Matriz de trazabilidad de los nueve compromisos, evidencias de pruebas y lista de escenarios pendientes.
- API publicada en Railway y respaldo web en Vercel, con comprobaciones HTTP.
- APK privado 0.7.0-trial, código 12, con firma, versión, Hermes, Firebase y hash comprobados.
- Separación de configuración pública móvil y secretos de backend; instrucciones de instalación y ejecución en los repositorios.

Qué falta para cerrarla

Completar aceptación física y recorrido de Auth, corregir hallazgos y cerrar evidencias académicas. Mantener explícita la limitación del histórico SQL inicial incompleto y preparar un corte revisado de entrega.

Por qué no se considera completada

La documentación y el paquete privado existen; falta evidencia de funcionamiento final y cierre académico. La firma de prueba no acredita una publicación en tienda.

Referencias principales de código y documentación

docs/architecture.md

docs/compliance-current-20261020.md

docs/permissions-recurrence-evidence-20261020.json

docs/android-apk-0.7.0-20261020.json

README.md

04 / DOCUMENTACIÓN CONSOLIDADA

Arquitectura, navegación y datos

Fuentes consolidadas: [F02]

Aplicaciones y responsabilidades

Este repositorio contiene dos aplicaciones independientes:

- apps/api: NestJS, autenticación de peticiones, reglas de viajes y reservas, Google Maps, operación del piloto y administración.
- apps/web: Next.js, sitio público, panel administrativo y páginas de respaldo de enlaces móviles. El login administrativo está integrado; la API comprueba el rol vigente y la cuenta, además de las protecciones de interfaz.

El repositorio separado krow-app-mobile contiene React Native 0.84, React Navigation y React Query. La app usa Supabase directamente para Auth; los datos de negocio se consultan y modifican por la API. Los paquetes de packages/ comparten contratos y acceso HTTP dentro del monorepo del backend/web.

Supabase aloja Auth, PostgreSQL y Storage. La API usa clientes con identidad del usuario o el LOGIN privado limitado del piloto, según el módulo. Este LOGIN usa TLS y no es superusuario ni tiene bypass de RLS. Las mutaciones de publicación/reserva validan en servidor y en funciones/transacciones SQL; los controles de pantalla no sustituyen esas reglas.

Navegación y estado móvil

RootNavigator recupera la sesión, atiende recuperación de acceso y separa autenticación de navegación principal. El acceso de conductor conserva su comprobación específica. Los stacks/tabs conducen a publicación, búsqueda, reservas, perfil, viajes programados, seguimiento e historial.

React Query administra consultas, invalidación y refrescos; los formularios y estados de interacción usan hooks locales. No se agregó Zustand ni Redux. useCurrentUserRole combina identidad de sesión, perfil del servidor y modo de acceso. Cambiar de cuenta o salir limpia consultas y detiene seguimiento.

Los enlaces HTTPS solo aceptan el origen aprobado y UUID de viaje válido. Un bookingId único y válido abre la bandeja en la solicitud concreta después de autenticación y habilitación del stack. La API vuelve a comprobar participación y estado. Las páginas web no consultan ni exponen detalles de viaje. Contrato de enlaces y flujo de solicitudes.

Un marcador único invite=1 abre la vista mínima de invitación tras login. El pasajero elige paradas y solicita un lugar mediante la API existente; el conductor aprueba. No concede acceso al roster, chat o GPS. Un propietario o participante abre su detalle protegido. Las invitaciones son enlaces compartidos, según la decisión del usuario; no se creó un sistema de invitaciones dirigidas a cuentas.

Cuando no hay viaje activo, la API responde HTTP 204 y el cliente devuelve null sin parsear JSON. Una ausencia normal no equivale a un fallo de red/servidor. Las salidas programadas pasadas no se presentan como próximo viaje activo.

Datos y conectividad

La app aplica timeout y cancelación a HTTP. expectedActorId vincula lecturas protegidas a la cuenta y queda fuera de la petición HTTP. La publicación usa una identidad persistida en Keychain/Keystore y un comprobante SQL para recuperar reintentos sin crear otro viaje.

La consulta parcial sin conexión guarda perfil y primera página de listas propias, por cuenta/API/grupo, con caducidad de lectura de 24 horas. Usa Keychain/Keystore, metadatos de consulta y revalidación; no concede

habilitación para publicar ni abre acciones operativas desde copias. Requiere una sesión local utilizable. No hay cola offline de escrituras del piloto. AsyncStorage sigue destinado a preferencias y otros identificadores locales existentes.

El servidor resuelve la autoridad y concurrencia de reservas, cupo, estados y cobros. Ninguna copia local se considera confirmación de una operación remota.

Mapas, seguimiento y eventos

La app representa mapas con las integraciones nativas existentes; la API consulta Google Maps para cálculo de rutas. El piloto utiliza avenidas y paradas publicadas por el conductor y búsqueda por destino del pasajero. Los filtros de fecha transmiten instantes UTC. La recurrencia publica un lote de hasta cuatro semanas con días seleccionados, vista previa y confirmación. Cada salida usa la API/idempotencia vigente y se administra por separado. El lote conserva identificadores y avance en Keychain/Keystore por cuenta/API para retomarlo explícitamente después de un fallo; no es creación indefinida ni escritura offline automática.

El seguimiento Android utiliza el servicio nativo y las sesiones de carga de ubicación. La API ofrece instantáneas/eventos del piloto. El runtime avanzado de Mapbox/v2 permanece separado mediante flags; estas etapas no lo activaron ni certifican su navegación offline.

El outbox y proveedor FCM distribuyen mensajes data; el receptor Android comprueba destinatario y caducidad. Firebase krowapp ya está configurado y los flags push están habilitados para aceptación privada en APK 0.7.0. Al abrir la app en primer plano se solicita una vez el permiso Android 13+, sin exigir login; después se registra el dispositivo automáticamente con sesión y permiso. La negativa no bloquea la app ni genera otra petición automática; Perfil conserva activación y ajustes. La API FCM aprobó una petición validate_only sin entrega; recepción/apertura física permanecen pendientes. Firebase no sustituye Supabase Auth o PostgreSQL.

El worker existente agrega recordatorios para conductor y pasajeros confirmados dentro de los 30 minutos previos a la salida. Usa el mismo outbox, clave única por salida/destinatario y comprobación de vigencia antes de enviar. Los avisos caducan al partir y no se envían si el viaje o reserva perdió vigencia. No se añadió un cron externo ni una alarma exacta local.

Las solicitudes y respuestas se recuperan por una consulta compartida de React Query cada tres segundos en primer plano y al reanudar; push no es la única fuente de recuperación. booking-inbox consulta estado vigente y participación, y reutiliza los eventos del outbox sin depender de su estado de entrega. Las confirmaciones vistas se guardan por cuenta/API en Keychain/Keystore; no son reconocimientos sincronizados entre teléfonos. No se añadió otro WebSocket.

El chat conserva paginación y reintentos idempotentes, con una lista cronológica sin transformaciones de inversión. El scroll sigue los mensajes nuevos únicamente cerca del final y conserva la posición al cargar anteriores. La estabilidad visual del teclado requiere aceptación nativa.

05 / DOCUMENTACIÓN CONSOLIDADA

Solicitudes, confirmaciones y chat

Fuentes consolidadas: [F04], actualizado con [F03]

Viaje activo. La API podía resolver null cuando no había viaje; NestJS enviaba un cuerpo vacío y el cliente intentaba interpretarlo como JSON. Se definió HTTP 204 para este caso y el cliente lo convierte en null sin parsear. Los errores reales de red/servidor siguen siendo errores. Las consultas se vinculan a la cuenta que las inició; se excluyen salidas ya pasadas, salvo viajes en curso. Las pruebas HTTP distinguen explícitamente 204 vacío, 200 con viaje y acceso no autorizado.

Solicitudes recuperables. GET /v1/booking-inbox devuelve hasta 50 solicitudes/respuestas propias. GET /v1/booking-inbox/:bookingId permite abrir la solicitud exacta y consultar su estado actual, incluso si fue cancelada, rechazada o venció su disponibilidad. Las consultas comprueban participación y cuenta activa; no se abre acceso directo a tablas ni se añaden migraciones.

La bandeja incluye pendientes del conductor de viajes futuros, y respuestas vigentes del pasajero identificadas por el outbox de los últimos siete días. No depende de que el aviso haya sido entregado: un registro pendiente, fallido o expirado del proveedor no consume la respuesta de la aplicación. Una confirmación antigua ya sustituida por cancelación no vuelve como si siguiera vigente. El historial mantiene el resto de las reservas.

Presentación. Se eligió una pantalla dedicada para revisar y responder, un acceso prioritario en Inicio y una hoja animada reutilizando AnimatedModal para destacar novedades en Inicio, Viajes o viaje programado. La hoja se puede posponer; no invade chat, formularios ni seguimiento activo. Muestra participante, subida, bajada, ruta, fecha, lugares, aportación y vehículo del conductor para el pasajero. Aceptar/rechazar tienen acciones separadas y bloqueo inmediato contra doble pulsación. Los colores provienen del tema; la animación existente respeta movimiento reducido.

Sincronización. Una consulta React Query compartida cada tres segundos en primer plano y una revalidación al reanudar recuperan la verdad del servidor. Se reutilizan invalidaciones de listas/detalles; no se añade WebSocket ni librería. Una red fallida conserva información para lectura, pero deshabilita acciones sobre resúmenes desactualizados. Antes de responder se vuelve a consultar la solicitud y el servidor valida de nuevo bajo bloqueo transaccional. Aceptar nuevamente el mismo estado es idempotente y no consume otro asiento; una cancelación o respuesta incompatible devuelve conflicto.

Confirmaciones vistas. Solo identificadores de evento y fechas se guardan en Keychain/Keystore, separados por cuenta/API. La escritura se serializa para no perder reconocimientos concurrentes. Se consideran vistos al pulsar Entendido o abrir el detalle; recibir un push o consultar datos no los consume. Revisar después solo pospone la hoja durante esa sesión. Persistencia por ocho días frente a la ventana remota de siete días; no es sincronización de leídos entre varios teléfonos. Un almacenamiento corrupto impide la alerta automática y conserva la bandeja consultable.

Push. Se reutilizan outbox, cifrado de tokens y proveedor FCM. Antes de enviar un evento de reserva se comprueba que su estado siga vigente. El payload incluye bookingId; Android incorpora ese UUID en el enlace de las notificaciones de reserva. La app espera sesión y acceso de conductor antes de abrir BookingRequests; el detalle vuelve a autorizar la participación. El receptor exige destinatario coincidente, UUID válidos y caducidad. Los avisos de chat/proximidad conservan la apertura de viaje existente.

Chat. Se eliminó inverted; los mensajes se ordenan cronológicamente y los recientes quedan abajo. La carga de anteriores es explícita en la cabecera y conserva la posición visible. El scroll solo sigue el final cuando el usuario está cerca de él; abrir el teclado mantiene ese criterio. Se conserva alineación por emisor, temas y reintento con el mismo clientId. Lecturas/envíos se vinculan al actor; una respuesta tardía al cambiar de conversación/cuenta no borra otro borrador ni reinicia su consulta.

Recorrido conductor-pasajero

- El pasajero solicita un lugar con subida y bajada explícitas.
- El conductor recupera la solicitud desde el backend; no depende exclusivamente del push.
- La bandeja y los avisos destacados permiten revisar datos y aceptar o rechazar.
- El cliente consulta el estado actual y el servidor valida capacidad, participación y concurrencia.
- El pasajero recupera la respuesta y las pantallas relacionadas se actualizan.
- Abrir una notificación consulta el estado vigente de la reserva, incluso si ya cambió.

> **Vigencia de esta sección**
> En 0.7.0 el permiso se pide al abrir y el registro push se intenta automáticamente tras login si está concedido. Ya no es obligatorio entrar primero a Perfil para habilitar el registro.

Contratos utilizados

Contrato	Uso
GET /v1/booking-inbox	Pendientes y respuestas propias. Hasta 50 elementos; consulta autorizada por cuenta.
GET /v1/booking-inbox/:bookingId	Estado actual de una reserva concreta; no permite consultar reservas ajenas.
GET /v1/rides/mine/active	HTTP 204 cuando no hay viaje activo; no se intenta interpretar un cuerpo vacío como JSON.

06 / DOCUMENTACIÓN CONSOLIDADA

Programación, recurrencia e invitaciones

Fuentes consolidadas: [F03], [F07], [F08]

Publicación recurrente

El usuario eligió publicar varias fechas con vista previa y confirmación. En Publicar viaje, el conductor selecciona fecha inicial/hora, días de la semana y un periodo de una a cuatro semanas. La revisión enumera las salidas reales y el botón confirma cuántos viajes se publicarán. La recurrencia conserva la hora de Ciudad de México independientemente de la zona del teléfono, incluida una semana que atraviese cambio de mes.

Cada fecha reutiliza el contrato de publicación vigente, el cálculo de ruta del servidor y su idempotencia. Antes de enviar la primera petición, publicationBatch guarda en Keychain/Keystore el lote y un UUID diferente por salida. La ejecución es secuencial, requiere conexión y se inicia explícitamente; no crea viajes en segundo plano.

Si falla una petición, los viajes ya confirmados se conservan. La pantalla ofrece retomar el lote desde el primer paso, incluso tras reiniciar el proceso, y usa los mismos identificadores para recuperar una respuesta perdida. Un lote incompleto no se sustituye silenciosamente por otro formulario. Finalizar el intento descarta las fechas restantes y conserva los viajes existentes; el diálogo pide revisar Mis viajes si alguna respuesta se perdió. Al retomar, las fechas fuera del margen de 15 minutos se omiten y se informa que deben revisarse.

Límites deliberados: hasta 28 salidas, cada viaje se administra por separado y no existe creación indefinida ni edición masiva de una serie. No se promete éxito atómico de todo el lote. El servidor continúa comprobando cuenta, vehículo, catálogo, permisos, fecha y cupo de cada publicación.

Invitación mediante enlace compartido

El usuario eligió enlace compartido con solicitud y aprobación del conductor. El botón **Invitar a este viaje** abre el panel nativo de compartir desde un viaje futuro del conductor. El enlace conserva el origen aprobado y añade un marcador único invite=1; el respaldo web solo conserva ese marcador válido, sin mezclarlo con un destino de reserva.

Después de iniciar sesión, el enlace abre RideInvitation. GET /v1/rides/:rideId/invitation devuelve una vista mínima a cuentas activas: recorrido, horario, nombre del conductor, descripción del vehículo, aportación, cupo y paradas activas. No devuelve lista de pasajeros, matrículas, chat, teléfonos ni GPS. Un viaje cancelado, pasado o iniciado deja de admitir una invitación para un tercero. Los participantes conservan acceso a su detalle protegido.

El pasajero debe seleccionar subida y bajada en orden y pulsar **Solicitar un lugar**. El cliente revalida antes de enviar y usa la API de reservas existente, vinculada a su cuenta. La reserva queda pendiente; el conductor la revisa en la bandeja ya implementada. Abrir el enlace no reserva, no acepta ni incrementa cupo. Si ya existe una reserva activa, se abre su detalle en lugar de insertar otra; una reserva cancelada no impide solicitar de nuevo si el viaje sigue disponible. En esta versión la invitación solicita un lugar por operación, en consonancia con el flujo elegido.

El enlace se puede reenviar: no está dirigido a una cuenta ni es una autorización para ver datos operativos. No se añadió un modelo de invitaciones privadas, conforme a la elección del usuario.

Búsqueda por fechas

RequestRideScreen permite abrir un filtro opcional de salida desde/hasta, elegir uno o ambos extremos y quitarlo. Reutiliza RideDateTimePicker, los botones y el tema existentes. El selector y el resumen usan la hora local del dispositivo; rideApi transmite los instantes en ISO UTC mediante el contrato existente fromTime/toTime.

Se validan fechas inválidas, un extremo final anterior al inicial y rangos que ya terminaron. Cambiar fechas descarta los resultados anteriores e invalida respuestas todavía en vuelo. Refrescar y reintentar conservan el rango. Si el rango vence mientras se consultan resultados, el refresco vuelve al formulario y retira la lista antigua. Quitar el filtro restaura la consulta de próximas salidas.

La API y SQL ya admitían ambos extremos opcionales y límites inclusivos. La búsqueda conserva la selección por destino/paradas publicadas y el orden por cercanía; el filtro no crea recorridos ni disponibilidad. El servidor sigue excluyendo salidas pasadas aunque el inicio indicado haya quedado atrás. No se agregaron dependencias ni migraciones.

Los filtros se conservan en 0.7.0. Las búsquedas válidas contra la API publicada retornaron listas vacías; los límites y escenarios con resultados se probaron en SQL aislado. Falta aceptación visual del selector en un teléfono. La pausa histórica de Railway ya no describe el despliegue actual.

07 / DOCUMENTACIÓN CONSOLIDADA

Lectura sin conexión y recuperación

Fuentes consolidadas: [F06], [F03]

La app conserva React Query, la API NestJS y Keychain/Keystore. No incorpora una nueva librería ni una cola de escrituras del piloto.

Información disponible

- Perfil previamente consultado, asociado a una sesión local existente.
- Primera página de Tus viajes: hasta 30 resúmenes por rol y grupo (próximos, en curso e historial).
- Datos guardados durante un máximo de 24 horas para lectura. El límite de tamaño por registro es 48.000 caracteres; las copias mayores no se persisten.

El espacio de almacenamiento incluye la URL de API, la cuenta y la consulta. Solo se guardan campos explícitos del perfil y de los resúmenes; no se persisten tokens, chat, GPS, rutas operativas ni detalles de pasajeros mediante este mecanismo. Las copias de negocio se almacenan con la protección nativa de Keychain/Keystore y sin respaldo en AsyncStorage.

Comportamiento

Primero se intenta obtener la respuesta del servidor. Solo una pérdida de conexión con la API (código de cliente 0) o un timeout (408) permite recuperar la copia protegida. Puede ser necesario esperar el timeout antes de mostrarla; no es una hidratación inmediata de toda la app. HTTP 401/403 borra las copias de esa cuenta; otros errores HTTP y cancelaciones se muestran sin recuperar la copia del disco.

Los resúmenes recuperados indican su fecha y advierten que el estado puede haber cambiado. No abren detalles ni pantallas operativas mientras la lista o el perfil proceden de una copia. Un perfil recuperado nunca concede habilitación para publicar. La pantalla conserva su diseño y componentes actuales.

Mientras se usa una vista recuperada, se consulta de nuevo cada 15 segundos en primer plano. Se revalida también al volver a la app y existe actualización manual. Una respuesta válida reemplaza el conjunto guardado con el estado del servidor: no se fusionan cambios locales ni se repiten escrituras. La paginación adicional requiere conexión.

La sesión sigue gestionada por Supabase. Este trabajo no habilita iniciar sesión sin conexión ni recuperar una sesión vencida que necesite renovación remota. Una escritura local fallida no interrumpe una consulta online correcta, pero no crea una copia alternativa sin protección. Si no hay una copia válida o falla su lectura, la consulta offline conserva el flujo de error/reintento.

Consistencia y privacidad

apiRequest.expectedActorId vincula estas lecturas a la cuenta de la sesión y nunca se envía como campo HTTP. Evita guardar una respuesta de otra cuenta tras un cambio de sesión. Cerrar sesión o cambiar de cuenta cancela la validez de operaciones de caché pendientes y borra las copias de la cuenta anterior; una respuesta tardía no las recrea.

Crear o aceptar una reserva, publicar/editar un viaje, cobrar, cancelar, enviar chat o cambiar estados sigue requiriendo confirmación del backend. No hay sincronización de escrituras offline y, por tanto, tampoco conflictos de escritura local/remota que resolver. La idempotencia de publicación de la etapa anterior sigue siendo independiente.

Qué ya se verificó y qué falta

Las pruebas automatizadas cubren recuperación con memoria nueva y recarga de módulos, separación por cuenta/consulta, caducidad, corrupción, fallos de almacenamiento, autorización denegada, respuesta tardía tras logout y sustitución al reconectar. Keychain/Keystore se simula en Jest: falta modo avión y recuperación en Android real.

Operación	Sin conexión
Perfil y listas ya consultadas	Lectura limitada con sesión local utilizable, copia válida y advertencia de vigencia.
Publicar, reservar, responder o cancelar	Requieren conexión y confirmación del backend.
Chat, cobros y cambios de estado	Requieren conexión. No se sincronizan escrituras offline automáticamente.
Lote recurrente interrumpido	Se conserva el avance; se retoma por acción explícita cuando hay conexión.
Mapas descargados y navegación offline	Fuera de la certificación de esta entrega; runtime avanzado desactivado.

08 / DOCUMENTACIÓN CONSOLIDADA

Notificaciones, Firebase y recordatorios

Fuentes consolidadas: [F03], [F05], [F13]

Permiso y registro de notificaciones

useNotificationPermissionStartup solicita el permiso al abrir la aplicación en primer plano, antes del login. En Android 13 o superior utiliza el diálogo de POST_NOTIFICATIONS. Guarda únicamente el hecho de haberlo solicitado en AsyncStorage para respetar la negativa o el descarte y no repetir la petición en cada apertura. Un fallo nativo sin Activity permite otro intento en un arranque posterior. Android anterior a 13 usa el estado de avisos del sistema y no tiene ese diálogo. iOS no ejecuta llamadas Android y no se declara implementado su registro nativo.

La autenticación continúa aunque se rechace el permiso. synchronizePilotPush registra el dispositivo automáticamente cuando hay sesión y permiso concedido; ya no exige entrar primero a Perfil. Retomar la app vuelve a comprobarlo, incluida una concesión desde Ajustes. Un fallo de red permite reintento en primer plano; una negativa no genera otra petición automática. Perfil conserva activación manual y acceso a los ajustes del sistema.

Las credenciales y la petición quedan vinculadas a la cuenta que inició el registro. Cambiar de cuenta o salir desactiva inmediatamente los avisos anteriores y cancela la petición pendiente. Renovar el registro de la misma cuenta conserva su activación nativa anterior si falla la red. El permiso de ubicación se mantiene independiente y comenzar GPS no vuelve a solicitar notificaciones.

Referencia de plataforma: permiso de notificaciones Android.

Recordatorios de salida

El worker existente del outbox identifica cada minuto viajes programados o llenos cuya salida se encuentra dentro de los siguientes 30 minutos. Añade un aviso al conductor y a pasajeros confirmados, con clave única por viaje, salida y destinatario. No genera recordatorios a pasajeros pendientes o cancelados. Antes de enviar vuelve a comprobar estado, fecha y participación; iniciar/cancelar el viaje o cancelar la reserva invalida un recordatorio anterior. La expiración es la hora de salida.

Se reutilizan cifrado de tokens, reintentos, leases, deduplicación y FCM. Android presenta el texto de recordatorio y abre el viaje existente. No se añadió un cron de infraestructura, alarma local ni servicio externo. La entrega depende de conexión, permiso, registro y restricciones del sistema; no es una alarma exacta ni certifica recepción con cierre forzado.

Estado actual de Firebase

Elemento	Estado documentado
Proyecto	krowapp; configuración Android del paquete com.krownmobileapp.
Cliente Android	google-services.json local ignorado por Git; el build valida paquete y recursos nativos.
Servidor	Credencial privada alojada en los secretos de NestJS/Railway. No se copia al móvil.
Flags	PILOT_PUSH_ENABLED=true y KROW_PILOT_PUSH_ENABLED=true para aceptación privada.
FCM HTTP v1	Petición validate_only aprobada con HTTP 200; cero mensajes entregados en esa comprobación.
Prueba física	Registro, recepción, presentación y apertura pendientes.

Push remoto y aviso local no son lo mismo

NestJS prepara el evento y lo envía por FCM como mensaje data. El servicio Android comprueba cuenta, identificadores y caducidad; después presenta un aviso en el teléfono. La presencia de un aviso local o una prueba con mocks no demuestra que el transporte remoto, la entrega en segundo plano y la apertura estén funcionando de extremo a extremo.

La bandeja recupera la verdad del backend de forma independiente. Por eso una solicitud puede seguir apareciendo aunque no llegue el push, el permiso esté denegado o el proveedor haya fallado.

Configuración nativa reutilizada

Variable nativa	Procedencia del JSON de cliente
KROW_FIREBASE_APP_ID	client[].client_info.mobilesdk_app_id
KROW_FIREBASE_API_KEY	client[].api_key[].current_key
KROW_FIREBASE_PROJECT_ID	project_info.project_id
KROW_FIREBASE_SENDER_ID	project_info.project_number

Los valores se exportan durante el build y se restauran después. La cuenta de servicio es rechazada por el importador. La inicialización programática existente usa FirebaseOptions; no se añadió otra librería ni un proyecto de datos Firebase.

09 / DOCUMENTACIÓN CONSOLIDADA

Enlaces y recuperación de cuenta

Fuentes consolidadas: [F03], [F07], [F09], [F12]

Un enlace lleva al usuario a una pantalla específica de KROW. El dominio aprobado es https://krow-web-gamma.vercel.app. La asociación Android y el respaldo web están publicados; el backend conserva las comprobaciones de acceso.

Destino	Forma del enlace	Comportamiento
Viaje	/rides/{UUID}	Abre el viaje y consulta lo que permite la cuenta.
Reserva	/rides/{UUID}?bookingId={UUID}	Abre la solicitud exacta y revalida participación/estado.
Invitación	/rides/{UUID}?invite=1	Abre la vista mínima para elegir paradas y solicitar un lugar.
Acceso	/auth/callback	Permite volver a KROW desde confirmación o recuperación; recorrido completo pendiente.

Reglas implementadas

- Solo se aceptan el origen HTTPS aprobado y UUID válidos.
- Los parámetros ambiguos se descartan; la invitación no se mezcla con un destino de reserva.
- Si falta sesión, se conserva el destino hasta completar el acceso.
- La página de respaldo no consulta detalles del viaje ni intercambia códigos de acceso en el navegador.
- El callback usa no-store, no-referrer y exclusión de indexado. La asociación usa el certificado de prueba actual.

Paso pendiente en Supabase Auth

El usuario indicó que agregará la siguiente dirección a Redirect URLs conservando las entradas existentes. No se ha confirmado todavía que esté guardada:

```
https://krow-web-gamma.vercel.app/auth/callback
```

Después debe probarse recuperación de contraseña y confirmación de correo desde una cuenta de prueba autorizada. Para el flujo PKCE se usa el mismo teléfono en el que se originó la solicitud. No se enviaron correos ni se certificó este recorrido durante la implementación.

Verificación Android prevista

Con el APK instalado y un Android conectado, los comandos documentados son:

```
adb shell pm verify-app-links --re-verify com.krownmobileapp
adb shell pm get-app-links com.krownmobileapp
```

La verificación debe cubrir aplicación abierta, segundo plano y proceso cerrado; también sesión cerrada y acceso de conductor pendiente. Estos comandos son instrucciones de aceptación, no resultados ejecutados en un teléfono.

10 / DOCUMENTACIÓN CONSOLIDADA

Instalación, ejecución y configuración

Fuentes consolidadas: [F05], [F14], [F15]

Repositorios y requisitos

Proyecto	Ubicación
API y web	C:/Users/Novasys/Documents/krow-app-backend
Móvil	C:/Users/Novasys/Documents/krow-app-mobile

El backend recomienda Node.js 24 o 22.22.2+ dentro de la rama 22, y npm 10+. El móvil declara Node.js 22.11.0 o superior y requiere un entorno React Native/Android preparado. La base actual es un proyecto Supabase existente; el histórico inicial de migraciones no permite declarar una reconstrucción completa desde cero.

API y web en desarrollo

Desde la raíz del backend, instalar dependencias y preparar los archivos locales a partir de sus ejemplos:

```
npm ci
apps/api/.env.example > apps/api/.env
apps/web/.env.example > apps/web/.env.local
```

Las dos líneas de archivos indican origen y destino de copia, no comandos de redirección. Completar los valores autorizados sin copiar secretos al código ni a este PDF.

```
npm run dev:api
npm run dev:web
```

Ejecutar cada aplicación en una terminal distinta. API local: http://localhost:3000/v1; health: /v1/health; Swagger: /v1/docs; web local: http://localhost:3001.

Móvil en desarrollo

```
npm ci
npm start
npm run android
```

Los comandos se ejecutan desde el repositorio móvil y requieren la configuración local y un Android/emulador disponible. La búsqueda del piloto parte del ITNL y no necesita pedir GPS al pasajero; el conductor autoriza ubicación al iniciar seguimiento.

Compilación privada Android preparada

El script existente usa .env.trial ignorado, la toolchain preparada y el JSON público de cliente Firebase. Para revisar la configuración sin compilar:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
scripts/build-android-trial.ps1
-FirebaseConfigFile android/app/google-services.json -ValidateOnly
```

El comando se presenta en varias líneas para lectura: debe ejecutarse como una sola línea. Para compilar, omitir -ValidateOnly. Bypass se limita a ese proceso y no cambia la política permanente de Windows.

Variables por responsabilidad

Ubicación	Nombres relevantes	Criterio
Móvil	SUPABASE_URL, SUPABASE_ANON_KEY, KROW_API_URL, KROW_LINK_ORIGIN, KROW_AUTH_REDIRECT_URL, flags piloto/GPS/push.	Solo configuración de cliente. KROW_API_URL usa el origen de la API sin añadir /v1.
API	SUPABASE_URL, SUPABASE_ANON_KEY, GOOGLE_MAPS_API_KEY, PILOT_DATABASE_URL, PILOT_DATABASE_CA, flags de piloto/GPS/push.	Conexión privada limitada con TLS y autorización; secretos fuera del APK.
FCM servidor	FCM_PROJECT_ID, FCM_CLIENT_EMAIL, FCM_PRIVATE_KEY, PILOT_PUSH_ENCRYPTION_KEY.	La clave privada y el cifrado se mantienen en el servidor.
Mapas Android	KROW_ANDROID_MAPS_API_KEY.	Clave nativa restringida al paquete/firma; distinta de la clave REST del backend.

Una modificación de recursos Firebase o mapas nativos requiere recompilar e instalar. El runtime avanzado permanece desactivado; no se necesitan servicios nuevos para los flujos de esta etapa.

11 / DOCUMENTACIÓN CONSOLIDADA

Pruebas, comprobaciones y límites

Fuentes consolidadas: [F09], [F10], [F11], [F12], [F13]

Resultados automatizados registrados

Ámbito	Resultado	Qué acredita
API	197 pruebas / 28 suites	Reglas, SQL aislado y concurrencia con PostgreSQL temporal. Se detuvo al terminar.
HTTP E2E	9 pruebas / 2 suites	Controladores, validación y SQL reales; Auth y Maps usan dobles de prueba.
Móvil	269 pruebas / 44 suites	Componentes, navegación, recuperación, estados, permisos y almacenamiento simulados.
Web	57 pruebas / 12 archivos	Respaldo y preservación segura de destinos, incluida invitación.
Calidad y build	Tipos, lint y compilaciones aprobados	API compilada y Android generado. Lint sin errores no significa cero advertencias.

La regresión API adicional de 27 pruebas corresponde a un subconjunto de la batería; no debe sumarse como cobertura independiente. Tampoco se suman reejecuciones móviles como si fueran casos nuevos.

Comprobaciones con servicios reales

Comprobación	Resultado	Límite
API publicada	18 comprobaciones HTTP aprobadas.	Sin crear viajes o reservas de prueba ni enviar notificaciones; publicación positiva/replay HTTP no certificados por esta lectura.
Supabase	5 comprobaciones con LOGIN limitado; transacción revertida.	Compatibilidad de consulta de invitación/outbox. No hubo intents persistidos de prueba.
Web pública	3 comprobaciones: invitación, asociación Android y callback HTTP 200.	No acredita hidratación de navegador ni asociación/apertura del SO.
FCM	HTTP 200 con validate_only.	Valida acceso y petición sin entrega. Cero notificaciones enviadas por esta prueba.
APK	Firma, versión, Hermes, Firebase y flags comprobados.	No instalado. ADB no detectó dispositivos.

Las 18 comprobaciones HTTP publicadas

Caso	HTTP	Resultado
Disponibilidad pública	200	Aprobó
Readiness y base del piloto	200	Aprobó
Perfil requiere autenticación	401	Aprobó
Publicación requiere autenticación	401	Aprobó
Perfil autenticado	200	Aprobó
Catálogo de avenidas autenticado	200	Aprobó
Bandeja autenticada	200	Aprobó
Reserva ajena protegida	404	Aprobó
Bandeja requiere autenticación	401	Aprobó
Invitación requiere autenticación	401	Aprobó
Invitación inexistente no disponible	404	Aprobó
Invitación real con vista mínima	200	Aprobó
Ausencia de viaje activo normal	204	Aprobó
UUID aceptado hasta validación de negocio sin creación	400	Aprobó
Validación UUID de publicación	400	Aprobó
Campos desconocidos rechazados	400	Aprobó
Búsqueda por fechas futuras	201	Aprobó
Fechas invertidas rechazadas	400	Aprobó

Interpretación de códigos y advertencias

Un 401 o 404 en los casos de protección es el resultado esperado; un 400 puede acreditar un rechazo correcto. Un HTTP 204 sin viaje es normal. Los contadores de pruebas no acreditan por sí solos uso físico, seguridad integral ni rendimiento medido.

La ejecución completa de lint móvil registró 383 advertencias en aquel corte; la revisión focalizada final no tuvo errores. Algunas suites existentes emitieron avisos act del renderer. Se documentan para no presentar una calidad sin advertencias que no se midió.

12 / DOCUMENTACIÓN CONSOLIDADA

Entrega publicada y paquete Android

Fuentes consolidadas: [F09], [F16]

Elemento	Referencia vigente
API	https://krow-api-production.up.railway.app/v1
Despliegue API	54fa0b9d-1d40-49c3-ae0a-6b1ef0b167cf · SUCCESS
Web	https://krow-web-gamma.vercel.app
Despliegue web	dpl_YdvLJ5xsU4UMjjrmBf7j7tQQdnGy · READY y promovido
Android	0.7.0-trial · código 12 · com.krownmobileapp
Tamaño	137 078 114 bytes
Arquitecturas	arm64-v8a, armeabi-v7a
Firma	Certificado existente de pruebas privadas; no es firma de distribución en Play Store.
Instalación física	No realizada; no había dispositivos ADB conectados.

Ubicación del APK

C:/Users/Novasys/Documents/krow-app-mobile/artifacts/android/krow-0.7.0-trial.apk

Huella SHA-256 del APK

2BCFC430CE425955CE4A9C0524279F8D
23F0CAAA384EE9A101411861F1DB05E1

La huella identifica el archivo y es distinta de las huellas del certificado de firma utilizadas por Firebase/Maps y la asociación Android.

Comprobaciones del paquete

- Firma verificada y certificado de pruebas esperado.
- Android mínimo API 24; APK no depurable y tráfico HTTP claro desactivado.
- JavaScript Hermes incluido; bibliotecas nativas ARM64 y ARMv7 presentes.
- Cuatro recursos Firebase contrastados con la configuración pública real, sin imprimir claves.
- Flag push habilitado en la compilación; funciones de permisos, recurrencia, invitación y recordatorios incluidas.

Condición de entrega

El paquete está preparado para pruebas privadas. No constituye aceptación de tienda, de iOS ni de funcionamiento físico certificado. Los despliegues y sus estados corresponden a la evidencia registrada el 20 de octubre de 2026; este PDF no consulta nuevamente su disponibilidad.

13 / DOCUMENTACIÓN CONSOLIDADA

Matriz de aceptación pendiente

Fuentes consolidadas: [F01], [F03], [F09]

Los siguientes casos deben producir evidencia real antes de cerrar los compromisos. Una prueba puede resolver pendientes de varias actividades sin repetirse por separado.

Prueba	Entorno	Resultado que se debe registrar
Permiso inicial	Un Android 13+; instalación/estado de permisos conocido.	Permitir, denegar, descartar; la app sigue operando y no repite el diálogo automáticamente.
Registro y retorno desde Ajustes	Android, sesión válida y conexión.	Registro tras login; revocar/conceder en Ajustes y volver a la app.
Cambio de cuenta	Un Android y dos cuentas autorizadas.	Ningún aviso de la cuenta anterior; peticiones antiguas no activan otro actor.
App Links	App abierta, segundo plano, cerrada y sin sesión.	Viaje, reserva e invitación abren el destino correcto y el servidor verifica estado.
Auth por correo	Callback guardado y cuenta de prueba autorizada.	Confirmación/recuperación, código válido/caducado y cambio de contraseña.
Lote recurrente	Conductor aprobado; fechas de prueba explícitas.	Vista previa, publicación del número esperado, pérdida de red y recuperación sin duplicados.
Invitación y respuesta	Dos participantes; dos instalaciones para recepción simultánea.	Solicitud pendiente, aprobación/rechazo, cupo único y confirmación consistente.
Push y recordatorio	Android con FCM registrado y viaje controlado.	Recepción/presentación/apertura en cada estado; no enviar aviso obsoleto al cancelar/iniciar.
Offline	Modo avión, almacenamiento real y posterior reconexión.	Copia válida identificada; bloqueo de acciones; revalidación, caducidad y separación de cuenta.
GPS y mapas	Dos participantes y recorrido controlado.	Ubicación, paradas, retrasos, conectividad, cambios de estado y finalización.
Chat y accesibilidad	Teclado, temas, texto ampliado y movimiento reducido.	Orden correcto, scroll estable, controles identificables y ausencia de recortes/bloqueos.
Rendimiento	Android representativo durante flujos reales.	Mediciones de navegación, memoria, listas y mapas; incidentes reproducibles.

Formato mínimo para registrar cada caso

- Versión y código del APK, modelo de teléfono y versión Android.
- Fecha, red, estado del permiso y cuentas de prueba anonimizadas.
- Pasos, resultado esperado y resultado observado, con evidencia.
- Incidente, gravedad, reproducción, corrección y comprobación posterior cuando aplique.

No incorporar secretos o datos personales a evidencias públicas. El cierre forzado por el usuario tiene restricciones del sistema: no se promete recepción push garantizada.

14 / DOCUMENTACIÓN CONSOLIDADA

Orden recomendado y cierre académico

Fuentes consolidadas: [F01], [F03], [F09]

Orden	Trabajo obligatorio	Por qué va aquí
1	Confirmar callback de Auth e instalar el APK final en Android.	Habilita aceptación de enlaces, permisos y sesión con la versión correcta.
2	Permisos, registro, cambio de cuenta y enlaces en un teléfono.	Permite resolver temprano bloqueos de acceso y privacidad antes del ensayo compartido.
3	Modo avión, persistencia y lote recurrente controlado.	Comprueba recuperación y ausencia de duplicados con almacenamiento nativo.
4	Invitación, solicitud, aceptación/rechazo y push entre dos participantes.	Cierra el flujo principal conductor-pasajero y sus notificaciones.
5	Recorrido GPS y mapas; chat, accesibilidad y rendimiento.	Valida el comportamiento operativo y la estabilidad real.
6	Corregir hallazgos, repetir casos afectados y registrar aceptación.	Cierra evidencias por requisito sin confundir código preparado con funcionamiento aprobado.

Qué no requiere nuevo desarrollo por defecto

La recurrencia finita, las invitaciones compartidas, los recordatorios y el permiso inicial ya se implementaron según las decisiones del usuario. No deben aparecer otra vez como funcionalidades inexistentes. Falta aceptarlas físicamente y corregir los problemas que se observen.

Viabilidad del cierre

Los pendientes descritos son técnicamente abordables con la arquitectura actual. No se identifica una imposibilidad que obligue a migrar React Query, reemplazar NestJS o mover la base fuera de Supabase. La ausencia de teléfono, segundo participante o configuración confirmada es una limitación de aceptación, no una imposibilidad técnica.

Límites y trabajo fuera de este cierre

- iOS sigue fuera del alcance nativo acordado; no se da por soportado por una compilación Android.
- Crear viajes indefinidamente, invitaciones privadas por cuenta y edición masiva de series no forman parte de las decisiones elegidas.
- Mapas descargados y escrituras offline generales no se requieren para la solución parcial acordada.
- Distribución pública en tienda requiere su propio proceso de firma/publicación; el APK actual es privado.
- El histórico SQL inicial incompleto limita reconstruir toda la base desde cero y debe conservarse como limitación de entrega.

> **Criterio de terminado**
> Cerrar cada actividad requiere evidencia de sus requisitos importantes, configuración de acceso confirmada, flujos principales aceptados y documentación de los hallazgos resueltos. El porcentaje estimado no sustituye esa aceptación.

15 / DOCUMENTACIÓN CONSOLIDADA

Fuentes, archivos y glosario

Fuentes consolidadas: Inventario documental de ambos repositorios

Las referencias [Fxx] remiten a los archivos locales siguientes. Se conservaron los nombres de componentes y contratos para permitir revisión de código. Los archivos originales permanecen sin modificación por esta consolidación.

[F01] compliance-current-20261020.md

Matriz vigente de nueve compromisos y aceptación.

C:/Users/Novasys/Documents/krow-app-backend/docs/compliance-current-20261020.md

[F02] architecture.md

Arquitectura y responsabilidades actualizadas.

C:/Users/Novasys/Documents/krow-app-backend/docs/architecture.md

[F03] permissions-recurrence-20261020.md

Etapa 0.7.0: permisos, recurrencia, invitaciones y recordatorios.

C:/Users/Novasys/Documents/krow-app-backend/docs/permissions-recurrence-20261020.md

[F04] booking-interactions-20261020.md

Solicitudes, confirmaciones y chat; etapa 0.6.0 normalizada al estado actual.

C:/Users/Novasys/Documents/krow-app-backend/docs/booking-interactions-20261020.md

[F05] firebase-android-setup.md

Configuración Firebase; se prioriza su estado reciente sobre apartados históricos.

C:/Users/Novasys/Documents/krow-app-mobile/docs/firebase-android-setup.md

[F06] offline-readonly.md

Consulta parcial offline, límites y pruebas.

C:/Users/Novasys/Documents/krow-app-mobile/docs/offline-readonly.md

[F07] mobile-links-android-20261020.md

Contrato de enlaces y aceptación Auth/Android; actualizado con 0.7.0.

C:/Users/Novasys/Documents/krow-app-backend/docs/mobile-links-android-20261020.md

[F08] search-date-filters-20261020.md

Filtros de búsqueda por fecha y su alcance.

C:/Users/Novasys/Documents/krow-app-mobile/docs/search-date-filters-20261020.md

[F09] permissions-recurrence-evidence-20261020.json

Evidencia conjunta final de API, web y APK.

C:/Users/Novasys/Documents/krow-app-backend/docs/permissions-recurrence-evidence-20261020.json

[F10] permissions-recurrence-http-20261020.json

Dieciocho comprobaciones de API publicada.

C:/Users/Novasys/Documents/krow-app-backend/docs/permissions-recurrence-http-20261020.json

[F11] permissions-recurrence-live-20261020.json

Cinco comprobaciones SQL con rol limitado y rollback.

C:/Users/Novasys/Documents/krow-app-backend/docs/permissions-recurrence-live-20261020.json

[F12] permissions-recurrence-web-20261020.json

Tres comprobaciones públicas web.

C:/Users/Novasys/Documents/krow-app-backend/docs/permissions-recurrence-web-20261020.json

[F13] booking-fcm-validation-20261020.json

FCM validate_only sin entrega.

C:/Users/Novasys/Documents/krow-app-backend/docs/booking-fcm-validation-20261020.json

[F14] README.md

Instalación y ejecución de API/web.

C:/Users/Novasys/Documents/krow-app-backend/README.md

[F15] README.md

Instalación y ejecución móvil; se priorizan instrucciones del alcance Android actual.

C:/Users/Novasys/Documents/krow-app-mobile/README.md

[F16] android-apk-0.7.0-20261020.json

Firma, recursos y metadatos del APK final.

C:/Users/Novasys/Documents/krow-app-mobile/docs/android-apk-0.7.0-20261020.json

[F17] permissions-recurrence-20261020.md

Guía de uso de la etapa Android 0.7.0.

C:/Users/Novasys/Documents/krow-app-mobile/docs/permissions-recurrence-20261020.md

Referencias de plataforma ya citadas en las guías

Permiso Android de notificaciones

Recepción FCM en Android

Contrato FCM validate_only

Verificación Android App Links

Redirect URLs de Supabase Auth

Estas referencias proceden de las guías y explican los contratos utilizados; no se consultaron de nuevo para generar este PDF.

Glosario breve

Término	Explicación sencilla
API / backend	Servicio que consulta datos y aplica las reglas de KROW.
Frontend	Aplicación y pantallas con las que interactúa el usuario.
Deep Link / App Link	Enlace que lleva a una pantalla concreta dentro de la app.
Push	Mensaje enviado desde el servidor a un dispositivo a través de FCM.
Outbox	Registro durable de avisos pendientes con reintentos y caducidad.
Idempotencia	Repetir la misma solicitud no vuelve a crear o consumir lo ya confirmado.
Keychain / Keystore	Protección nativa usada para guardar información local sensible.
Mock / doble de prueba	Sustitución controlada de una dependencia; no demuestra entrega física o GPS.
APK trial	Archivo Android firmado para pruebas privadas.